

LeMoonz Portfolio

Project Documentation & Migration Notes

Project stage:	Active development
Current version:	Node.js, Express, EJS, Vanilla JavaScript, CSS
Next frontend:	React + TypeScript
Backend direction:	Python, with FastAPI currently considered
Data direction:	PostgreSQL
Deployment:	Railway
Developer:	Daniel Abramek
Updated:	August 2026

What this document is

This is a practical snapshot of LeMoonz while it is still being built. It explains what works today, what I want to change next, why I am moving toward React and TypeScript, and what I want the finished project to become. Planned features are kept separate from the current implementation so the document does not pretend the project is further along than it really is.

The production repository is private because it contains client-specific content and original artwork. This public documentation focuses on the technical side of the project.

Contents

1	The project in a nutshell	2
2	Current prototype: Node.js, Express and EJS	2
2.1	Current stack	2
2.2	How the current version works	2
2.3	What the prototype already proves	2
3	Frontend and visual direction	3
4	Why I am moving to React and TypeScript	3
4.1	Why React	3
4.2	Why TypeScript	3
4.3	What I am not doing	4
5	How I plan to migrate it	4
6	Target frontend	4
6.1	Frontend priorities	5
7	Backend direction: Python, API and PostgreSQL	5
7.1	What the API would be responsible for	5
7.2	Why PostgreSQL	5
8	The finished project I am aiming for	5
9	Quality: accessibility, performance and security	6
9.1	Accessibility	6
9.2	Performance	6
9.3	Security and privacy	6
10	Testing and deployment	6
11	Roadmap	7
12	What this project is teaching me	7
13	Current vs next vs final	8
14	Closing note	8

1 The project in a nutshell

LeMoonz is a portfolio and commissions website I am building for an independent artist and animator. The goal is simple: give the artist one place that feels like her own work, presents selected projects clearly, explains commission options, works well on phones and desktops, and is easy to keep developing later.

The important part is that this is a **real client project**. The design changes through feedback, content changes, and technical decisions have to work with actual artwork instead of placeholder cards from a tutorial.

The current prototype is a server-rendered Node.js application using Express and EJS. The frontend uses semantic HTML, CSS and Vanilla JavaScript. The app is deployed through Railway and versioned with Git/GitHub.

NEXT - What I am changing

I am starting to move the frontend toward React and TypeScript. I also want a cleaner boundary between frontend and backend, with Python/FastAPI and PostgreSQL as the current target direction for the later backend and data layer.

TARGET - Where I want it to end up

A polished portfolio and commissions site with reusable React components, typed frontend data, a small API, structured content in PostgreSQL, responsive media-heavy pages and a deployment process that stays simple enough for one developer to maintain.

2 Current prototype: Node.js, Express and EJS

The first version was built with technologies I already understood well enough to work without hiding too much behind a framework. That was useful. It made me deal directly with routing, templates, page structure, responsive CSS and deployment.

2.1 Current stack

Runtime	Node.js
Server	Express
Templates	EJS
Frontend	HTML, CSS, Vanilla JavaScript
Version control	Git and GitHub
Hosting	Railway

2.2 How the current version works

The request flow is intentionally simple:



Express chooses the route, EJS builds the page on the server, and the browser receives ready HTML. CSS handles most of the visual work, while JavaScript is used only where the interface needs behaviour.

2.3 What the prototype already proves

The current version already gives me a working base for:

- the visual direction and typography,
- a responsive layout strategy,

- reusable EJS partials,
- routing and server-side rendering,
- a real Git workflow,
- deployment through Railway,
- working directly from client feedback.

This matters because the migration is not starting from a blank tutorial project. I already have a real interface and real requirements to compare against.

3 Frontend and visual direction

LeMoonz should feel more like an editorial art portfolio than a generic software landing page. The current visual direction uses cream or white surfaces, dark charcoal text, deep forest green and small warm accents. Typography is serif-led and intentionally warmer than a typical tech product.

Design rule

The website should support the artwork instead of competing with it. Layout, motion and colour can have personality, but the artist's work still has to remain the strongest thing on the page.

The current CSS is built around flexible layouts, relative units and responsive behaviour instead of hard-coded desktop-only sizes. The migration to React does not change this design goal. React is an implementation change, not a visual reset.

4 Why I am moving to React and TypeScript

The EJS version works, so the reason for moving is not “EJS bad, React good.” The reason is that the project is becoming a better place to learn and use a more modern frontend structure.

4.1 Why React

As the site grows, I want repeated interface pieces to become real reusable components instead of template fragments plus scattered JavaScript. A gallery item, navigation state, commission card or project detail can each have a clear component boundary.

React also makes more sense once the site starts needing richer interface state. Instead of manually finding DOM elements and changing them, the UI can be described from application state.

```
state changes
  -> component renders the new UI
  -> React updates what changed
```

That is the main mental shift I want to learn properly.

4.2 Why TypeScript

TypeScript is mainly about making assumptions visible. Portfolio data, API responses and component props can have defined shapes instead of relying on memory.

For example, a project can be described as:

```
type Project = {
  id: string;
  title: string;
  description: string;
  images: string[];
  tags: string[];
```

```
};
```

For a small project this may look like extra work. For a growing project it should make refactoring safer and mistakes easier to catch.

4.3 What I am not doing

I do not want to rewrite everything in one weekend just to be able to put React on the README. The point is to understand the new architecture while keeping the current project as a reference.

5 How I plan to migrate it

The migration is deliberately gradual.

1. **Keep the EJS version working.** It stays as the reference implementation until the replacement is ready.
2. **Build the React + TypeScript shell.** Set up routing, global styles and the main page structure.
3. **Move static sections first.** Header, bio, footer and portfolio cards are safer places to learn component structure.
4. **Move real data into typed models.** Stop hard-coding project data directly into views.
5. **Add an API only where it is useful.** Do not create backend complexity just because “full stack” sounds better.
6. **Introduce PostgreSQL when content actually benefits from persistence.**
7. **Replace the old version only after feature parity is good enough.**

Migration principle

Every new layer should solve a real problem or teach a skill I can explain. If I cannot explain why a piece exists, the architecture is probably getting ahead of me.

6 Target frontend

The target frontend is a React + TypeScript application organised around a small number of reusable sections rather than hundreds of tiny components.

A possible structure is:

```
src/  
  components/  
    Header/  
    ProjectCard/  
    CommissionCard/  
    ContactForm/  
  pages/  
    Home/  
    Portfolio/  
    Project/  
    Commissions/  
    Contact/  
  types/  
  api/  
  styles/
```

The first version does not need a large state-management library. Local component state and a small amount of shared state should be enough until the project proves otherwise.

6.1 Frontend priorities

- responsive layouts first,
- accessible navigation and forms,
- clear loading and error states,
- image-heavy pages that still feel fast,
- reusable components without abstracting everything too early,
- TypeScript types for data crossing component or API boundaries.

7 Backend direction: Python, API and PostgreSQL

The current application already has a Node/Express backend. The longer-term direction is to separate the frontend from a small Python API. FastAPI is currently the framework I want to explore because it fits the kind of typed, explicit API I want to learn.

This is a **target direction**, not a claim that the production project already runs on Python.

7.1 What the API would be responsible for

The backend should stay boring and useful:

- return portfolio/project data,
- provide commission information,
- handle contact-form submissions,
- validate incoming data,
- keep secrets outside frontend code,
- later support simple content management if the project actually needs it.

A small API might eventually expose endpoints such as:

```
GET /api/projects
GET /api/projects/{slug}
GET /api/commissions
POST /api/contact
```

7.2 Why PostgreSQL

PostgreSQL is the target database because the content is naturally structured: projects, images, categories, commission types and messages all have clear relationships. It also gives me a useful relational database to work with in a real web project instead of only in university exercises.

I do not need the database on day one of the React migration. Static typed data is fine until persistence provides an actual benefit.

8 The finished project I am aiming for

Version 1.0 does not need to be a giant platform. A good finished LeMoonz site should simply do its core job really well.

TARGET - Version 1.0

- polished responsive home and about sections,
- portfolio browsing with clear project pages,
- commission information presented in a way that is easy to update,
- a reliable contact path,
- strong mobile behaviour,
- accessible navigation and sensible keyboard use,
- optimized images and acceptable loading performance,
- React + TypeScript frontend,
- a small backend/API where dynamic data is useful,
- PostgreSQL only for data that genuinely belongs in a database,
- deployment that remains understandable and maintainable.

The end goal is not “use as many technologies as possible.” The end goal is a site the client is happy to use and a codebase I understand well enough to maintain.

9 Quality: accessibility, performance and security

These are not separate “enterprise” features to add at the end. They are basic requirements for a public website.

9.1 Accessibility

The project should keep semantic HTML, useful alt text, keyboard-friendly navigation, visible focus states, sensible heading order and form labels. React components must not replace the good parts of normal HTML with div-heavy custom controls.

9.2 Performance

Artwork is likely to be the heaviest part of the site. Images should therefore be resized appropriately, compressed, lazy-loaded where useful and served in modern formats when possible. JavaScript should stay proportional to the actual interface needs.

9.3 Security and privacy

The project should keep secrets in environment variables, validate contact-form input on the server, avoid exposing private client information and keep dependencies updated. If analytics are added later, they should be chosen with privacy in mind instead of added automatically.

10 Testing and deployment

I do not need an enterprise test suite for an early portfolio project, but I do want enough testing to stop obvious regressions as the migration gets more complicated.

The practical order is:

- manual checks while components are still moving quickly,
- unit/component tests for logic that is easy to break,
- API tests once the backend exists,
- a small end-to-end smoke test for the most important visitor flows.

Railway remains the current deployment platform. The target workflow is still simple:



GitHub Actions can be added for basic CI when there are enough tests and build checks to make the pipeline useful.

11 Roadmap

The roadmap is intentionally short. It is a direction, not a promise that every item will happen on a fixed date.

Now	Keep improving the EJS prototype, document current behaviour, prepare the React/TypeScript project and learn the component model on real sections.
Next	Port the main pages, build reusable components, move project content into typed data and reproduce the current responsive behaviour.
After that	Introduce a Python/FastAPI backend where it actually helps, connect PostgreSQL for structured content, add stronger validation and testing.
Version 1.0	Stable responsive portfolio, commissions information, contact flow, good accessibility/performance and a maintainable deployment.

12 What this project is teaching me

LeMoonz started as a way to build a useful website for someone. It has also become the project I use to move from “I can make a page work” toward understanding how a larger web application should be structured.

The main things I want to get better at are:

- component-based frontend thinking,
- TypeScript and explicit data models,
- separating frontend and backend responsibilities,
- designing small REST APIs,
- using PostgreSQL in a real application,
- testing and debugging migrations instead of rewriting blindly,
- making technical decisions I can actually explain,
- working with real client feedback without losing control of the codebase.

The rule I want to keep

Use tools and AI when I understand the problem well enough to review the result. If I cannot explain what the code is doing, I need to slow down and learn that part before adding more complexity.

13 Current vs next vs final

	Current	Next	Target
Frontend	EJS + HTML/CSS/JS	React + TypeScript migration	React + TypeScript production frontend
Backend	Node.js + Express	Keep current backend while learning	Small Python/FastAPI API where useful
Data	Mostly code/static content	Typed frontend data	PostgreSQL for structured persistent data
Deploy	Railway	Railway during migration	Simple automated deploy with checks
Project state	Working prototype	Incremental rebuild	Stable client-facing version 1.0

14 Closing note

LeMoonz is still a work in progress, and this document is meant to stay honest about that. The EJS version is the current working base. React and TypeScript are the next frontend step. Python/FastAPI and PostgreSQL are the current backend/data direction, not finished features.

The project will probably change again as I learn more and as the client's needs become clearer. That is expected. The goal is not to lock the architecture forever; it is to make each version more understandable, maintainable and useful than the one before it.